
boolean



JS - Objects

Seconda Parte

Objects

Qualche agevolazione

Se le chiavi corrispondono ai nomi delle variabili possiamo inserire direttamente
solo il nome della chiave

```
let name = 'Paolo';
let age = 30;
let email = 'paolo@email.it';

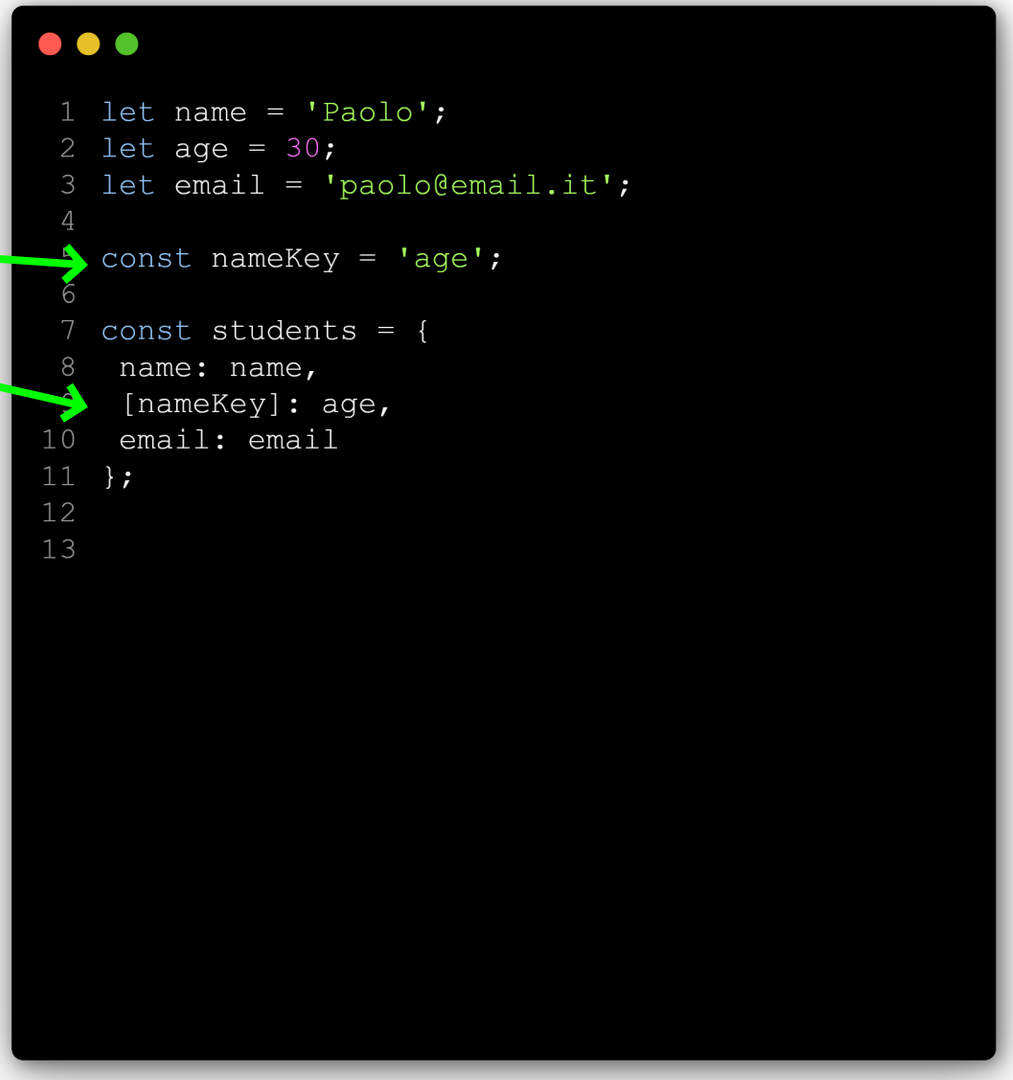
//es5 style sempre valido
const students = {
 name: name,
 age: age,
 email: email
};

//es6 style
const students = {
 name,
 age,
 email
};
```

Objects

Qualche agevolazione

Possiamo anche creare delle
chiavi dinamiche



```
let name = 'Paolo';
let age = 30;
let email = 'paolo@email.it';

const nameKey = 'age';

const students = {
 name: name,
 [nameKey]: age,
 email: email
};
```

Destructuring

Più difficile a dirsi che a farsi



Possiamo infine destrutturare...
Ma che significherà mai?

Destrutturare sta per **prendere alcune proprietà di un object e inserirle in variabili**, ma scrivendo solo una riga di codice.

Più difficile a dirsi che a farsi!

Destructuring

Più difficile a dirsi che a farsi



Solitamente come prendiamo una proprietà di un objects?

```
1 const student = {
2  name: 'Paolo',
3  age: 30,
4  email: 'paolo@email.it'
5 };
6
7 const name = ????
8
```

Destructuring

Più difficile a dirsi che a farsi

- **keyword**
  var, let o const per dichiarare le variabili
- **{NomeChiave}**
  Tante chiavi quante le proprietà che vogliamo estrapolare
- **=**
  Stiamo assegnando a queste variabili il valore che troveremo poi nell'object
- **Nome dell'object**
  da cui prendere i valori



```
1  const student = {
2   name: 'Paolo',
3   age: 30,
4   email: 'paolo@email.it'
5  };
6
7  const {name, email} = student;
8  console.log(name, email);
9  //Otteniamo: Paolo, paolo@email.it
10
```

Rest e Spread

A volte può essere utile conservare il **resto**, la parte rimanente, di una serie di elementi.
In questo esempio abbiamo una funzione che potrebbe ricevere un numero variabile di argomenti.

- **...**
  tre punti, operatore **Spread**
- **Nome parametro**
  Inseriamo un nostro nome per l'array che conterrà gli elementi

```
1 function myFunction(...myArguments) {
2  console.log(myArguments);
3 }
4 myFunction('uno', 3, {name: 'pippo'});
5 //avremo un array composto da tre elementi
```

L'unione dell'operatore Spread e il nome del parametro, prendono quindi il nome di **parametro Rest**.

L’operatore Spread

Copiamo e uniamo!

```
1 const obj = {name: 'palla', peso: 50};
2 const copyObj = {...obj, colore: 'blue'};
3 console.log(copyObj);
4 //{name: "palla", peso: 50, colore: "blue"}
```

L’operatore Spread può essere usato anche per copiare e/o unire parti di oggetti (e anche array).



LIVE CODING

ESERCITAZIONE